Bézier Curve: Quartic (3-D)

CairoMakie Visualisation

Code and Plot Explanation for Julia Beginners

`21-bezier-quartic-makie.jl`

E.R. Nurwijayadi

2026-03-15

Directed and curated by E.R. Nurwijayadi. Code and explanations AI-generated.

This document explains `21-bezier-quartic-makie.jl`. This is the first 3-D plot in the series. The curve is the same quartic Bézier (5 control points) used throughout scripts 01–05 for algebraic computation. New concepts: `Axis3` for 3-D axes, camera angles (`elevation`, `azimuth`), three-argument drawing functions, `Label` for figure-level text overlays, and generating a highlight list with `collect` + `round`..

Contents

1 The Plot **3**

2 Reading the Plot **3**
2.1 The 3-D space . . . 3
2.2 Visual elements . . . 4
2.3 The camera viewpoint . . . 4

3 Data and Constants **4**
3.1 Control points — the full matrix stored directly . . . 4
3.2 `U_HIGHLIGHT` — range to vector . . . 4
3.3 3-D offsets . . . 5

4 The `BezierQuartic` Struct and Math Functions **5**
4.1 Struct design: one matrix field . . . 5
4.2 `bq_point` — five Bernstein weights . . . 5

4.3 `bq_curve` and `bq_highlight` 6

5 `draw_and_save` — 3-D Plotting with Makie **7**
5.1 `Axis3` — the 3-D axis type 7
5.2 3-D drawing functions 8
5.3 3-D text annotations 9
5.4 `Label` — a figure-level text overlay 9
5.5 Legend 10

6 Type Map **11**

7 Entry Point **11**

1. The Plot

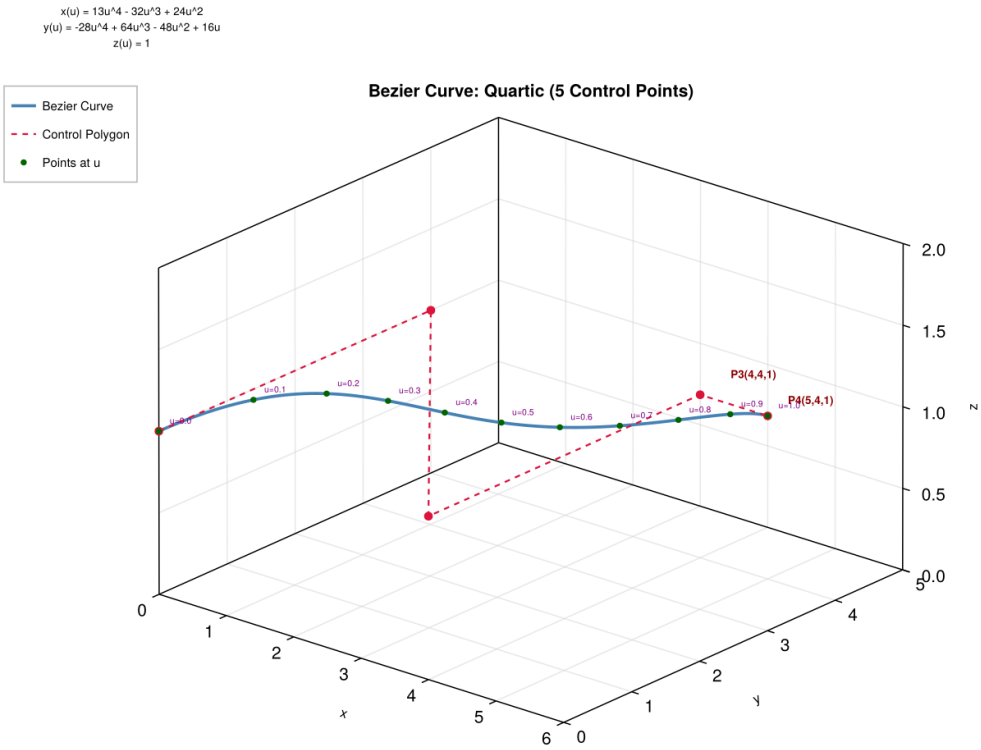


Figure 1: Quartic Bézier curve in 3-D space, with 5 control points and 11 highlighted curve points at $u = 0.0, 0.1, \ldots, 1.0$.

2. Reading the Plot

2.1. The 3-D space

This is the same quartic Bézier curve computed in scripts 01–05, now displayed in the full 3-D space (x, y, z). The five control points are:

Point	x	y	z
P_0	0	0	1
P_1	0	4	1
P_2	4	0	1
P_3	4	4	1
P_4	5	4	1

All five points have $z = 1$, so the curve lies entirely in the horizontal plane $z = 1$. In the 3-D view this appears as a flat curve hovering above the $z = 0$ floor.

2.2. Visual elements

Blue curve — the quartic Bézier, 300 sample points.

Red dashed polygon — the control polygon connecting $P_0 \to P_1 \to P_2 \to P_3 \to P_4$. Notice how P_1 and P_2 are the two "off-path" points — the polygon dips below into different y values and creates the characteristic crossing pattern visible in the 3-D view.

Green dots — 11 highlighted points at $u = 0.0, 0.1, 0.2, \ldots, 1.0$, each labelled with its u value.

Equation text (top-left) — the three parametric polynomials $x(u)$, $y(u)$, $z(u)$ displayed above the plot in monospace font. These are the same results computed in scripts 01–05.

2.3. The camera viewpoint

The plot is rendered from a specific 3-D camera angle. You can see the x axis running left-to-right along the bottom-front edge, the y axis running right-to-back, and the z axis vertical on the right. The viewpoint is set by `elevation` and `azimuth` in the code (explained in Section 5.1).

3. Data and Constants

3.1. Control points — the full matrix stored directly

```
const CONTROL_POINTS = [
    0.0  0.0  1.0;   # P0
    0.0  4.0  1.0;   # P1
    4.0  0.0  1.0;   # P2
    4.0  4.0  1.0;   # P3
    5.0  4.0  1.0;   # P4
]
```

A 5×3 matrix — five rows (one per point), three columns (x, y, z). This is the same control point data used in scripts 01–05 for algebraic computation, now stored directly as a matrix.

3.2. `U_HIGHLIGHT` — range to vector

```
const U_HIGHLIGHT = round.(collect(0.0:0.1:1.0), digits=2)
```

This one line packs three operations:

`0.0:0.1:1.0` — a **range** from 0 to 1 in steps of 0.1. In Julia, `a:step:b` creates a lazy range object — it does not allocate memory for all the values yet.

`collect(...)` — converts the lazy range into a concrete `Vector{Float64}`: `[0.0, 0.1, 0.2, 0.3, ..., 1.0]`.

`round.(..., digits=2)` — applies `round(x, digits=2)` to every element using broadcast. This is a safety measure: floating-point arithmetic can produce values like `0.30000000000000004` instead of `0.3` due to binary representation. Rounding to 2 decimal places cleans these up.

> **Result of `U_HIGHLIGHT`:**
>
> `[0.0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1.0]`
>
> 11 values. These produce the 11 green dots on the plot.

3.3. 3-D offsets

```
const CTRL_OFFSETS = [
    (-0.3, -0.4,  0.05),
    (-0.3,  0.3,  0.05),
    ( 0.15, -0.4, 0.05),
    ( 0.15,  0.3, 0.05),
    ( 0.15,  0.15, 0.05),
]
```

Each offset is now a **3-tuple** `(dx, dy, dz)` instead of a 2-tuple. The dz component is always `0.05` — just enough to lift the label slightly above the control point marker in the z direction so they don't overlap. The dx and dy values push labels away from the marker in the xy-plane.

4. The `BezierQuartic` Struct and Math Functions

4.1. Struct design: one matrix field

```
struct BezierQuartic
    points::Matrix{Float64}
end
```

Unlike `BezierQuadratic` (scripts 11–12) which stored three separate vector fields `p0`, `p1`, `p2`, this struct stores all five control points as a single `Matrix{Float64}` field.

For five points this is cleaner than five named fields. The trade-off: accessing a control point requires row-indexing `bz.points[i,:]` rather than `bz.p0`.

4.2. `bq_point` — five Bernstein weights

```
function bq_point(bz::BezierQuartic, u::Float64)
    v  = 1 - u
    b0 = v^4
    b1 = 4 * u * v^3
    b2 = 6 * u^2 * v^2
    b3 = 4 * u^3 * v
    b4 = u^4
    b0 .* bz.points[1,:] .+
    b1 .* bz.points[2,:] .+
```

```
    b2 .* bz.points[3,:] .+
    b3 .* bz.points[4,:] .+
    b4 .* bz.points[5,:]
end
```

`v = 1 - u` is precomputed to avoid repeating it five times. The five Bernstein basis values for degree 4 are then:

$$B_0 = v^4, \quad B_1 = 4uv^3, \quad B_2 = 6u^2v^2, \quad B_3 = 4u^3v, \quad B_4 = u^4$$

Each `bz.points[i,:]` is a row-slice: a length-3 vector `[x, y, z]` for control point P_{i-1}. The result is a length-3 vector — the (x, y, z) coordinates of the curve at u.

Content of `bq_point(bz, 0.5)`:

$v = 0.5$. Powers: $v^4 = 0.0625$, $uv^3 = 0.125$, etc.

Term	Value	$\times$ control point
$B_0 = 0.0625$		$\times P_0 = [0, 0, 1] \to [0, 0, 0.0625]$
$B_1 = 0.25$		$\times P_1 = [0, 4, 1] \to [0, 1.0, 0.25]$
$B_2 = 0.375$		$\times P_2 = [4, 0, 1] \to [1.5, 0, 0.375]$
$B_3 = 0.25$		$\times P_3 = [4, 4, 1] \to [1.0, 1.0, 0.25]$
$B_4 = 0.0625$		$\times P_4 = [5, 4, 1] \to [0.3125, 0.25, 0.0625]$

Sum: $[0 + 0 + 1.5 + 1.0 + 0.3125,\ 0 + 1.0 + 0 + 1.0 + 0.25,\ 0.0625 + 0.25 + 0.375 + 0.25 + 0.0625] = [2.8125,\ 2.25,\ 1.0]$

This matches the expected $P(0.5)$ from scripts 01–05. ✓

4.3. `bq_curve` and `bq_highlight`

```
function bq_curve(bz::BezierQuartic, n::Int=300)
    u_vals = LinRange(0.0, 1.0, n)
    hcat([bq_point(bz, u) for u in u_vals]...)'  # (n x 3)
end

function bq_highlight(bz::BezierQuartic, u_list::Vector{Float64})
    hcat([bq_point(bz, u) for u in u_list]...)'  # (m x 3)
end
```

Identical in pattern to `curve()` and `highlight_points()` from scripts 11–12, with the same `hcat + splat + transpose` idiom. The result is now $n \times 3$ (three coordinate columns) instead of $n \times 2$. `n = 300` (vs 200 in the quadratic scripts) gives a smoother curve for the more complex quartic shape.

Content of `cpts = bq_curve(bz)` (selected rows):

Row	x	y	z
1	0.0000	0.0000	1.0000
2	0.0067	0.0531	1.0000
⋮			
150	2.8125	2.2500	1.0000
⋮			
300	5.0000	4.0000	1.0000

Every row has $z = 1.000$ confirming the curve is flat in the z direction.

Content of `hpts = bq_highlight(bz, U_HIGHLIGHT)`:

Row (u)	x	y	z
1 ($u = 0.0$)	0.0000	0.0000	1.0000
2 ($u = 0.1$)	0.2196	0.3439	1.0000
3 ($u = 0.2$)	0.7168	0.9984	1.0000
4 ($u = 0.3$)	1.3104	1.6464	1.0000
5 ($u = 0.4$)	1.8944	2.0736	1.0000
6 ($u = 0.5$)	2.8125	2.2500	1.0000
7 ($u = 0.6$)	3.4944	2.3616	1.0000
8 ($u = 0.7$)	3.9424	2.7664	1.0000
9 ($u = 0.8$)	4.3136	3.2896	1.0000
10 ($u = 0.9$)	4.7856	3.7584	1.0000
11 ($u = 1.0$)	5.0000	4.0000	1.0000

5. `draw_and_save` — 3-D Plotting with Makie

5.1. `Axis3` — the 3-D axis type

```
ax = Axis3(fig[1, 1],
    title           = "Bezier Curve: Quartic (5 Control Points)",
    titlesize       = 14,
    titlegap        = 12,
    xlabel          = "x",
    ylabel          = "y",
    zlabel          = "z",
    xlabelsize      = 11,
    ylabelsize      = 11,
    zlabelsize      = 11,
    limits          = (0, 6, 0, 5, 0, 2),
    elevation       = deg2rad(22),
    azimuth         = deg2rad(360 - 50),
    backgroundcolor = :white,
    xgridcolor      = (:lightgray, 0.5),
    ygridcolor      = (:lightgray, 0.5),
    zgridcolor      = (:lightgray, 0.5),
```

```
)
```

`Axis3` is Makie's 3-D axis type. It replaces `Axis` whenever the plot has three spatial dimensions. Most attributes (`title`, `titlesize`, `titlegap`, `xlabel`, `limits`, grid colours) work the same as `Axis`, but there are three new ones.

`zlabel = "z"` and `zlabelsize`: The third axis label and its font size. In 2-D `Axis` there are only `xlabel` and `ylabel`. `Axis3` adds `zlabel`.

`limits = (0, 6, 0, 5, 0, 2)`: A 6-tuple: $(x_{\min}, x_{\max}, y_{\min}, y_{\max}, z_{\min}, z_{\max})$. In 2-D `Axis` the `limits` was a 4-tuple. Here the three axes each get their own min/max range.

`elevation` and `azimuth`: These control the camera viewpoint — where the virtual camera sits around the 3-D scene.

> **Understanding elevation and azimuth:**
>
> Imagine standing at the centre of the 3-D scene and looking at it from the outside. Your position is described by two angles:
>
> **Elevation** — how high up you are. 0 = looking from the equator (side view). 90° = looking straight down from above (top view). `deg2rad(22)` ≈ 0.384 radians puts the camera slightly above the horizontal — a low-angle view that shows the curve's flat $z = 1$ plane clearly.
>
> **Azimuth** — which direction around the scene you are standing. `deg2rad(360 - 50) = deg2rad(310)` rotates the camera 310° around the vertical axis, placing it at a viewpoint that shows the x-axis going left-to-right along the front edge and the y-axis going into the scene.
>
> **`deg2rad(x)`:** Converts degrees to radians. Julia's trigonometric functions work in radians. Writing `deg2rad(22)` is more readable than writing `22 * pi / 180`.

5.2. 3-D drawing functions

```julia
# Bezier curve
lines!(ax, cpts[:,1], cpts[:,2], cpts[:,3], ...)

# Control polygon line
lines!(ax, cp[:,1], cp[:,2], cp[:,3], ...)

# Control polygon markers
scatter!(ax, cp[:,1], cp[:,2], cp[:,3], ...)

# Highlighted points
scatter!(ax, hpts[:,1], hpts[:,2], hpts[:,3], ...)
```

Every drawing function now takes **three** coordinate arguments (`xs`, `ys`, `zs`) instead of two. This is the only mechanical change for going from 2-D to 3-D in Makie — the functions `lines!`, `scatter!`, and `text!` all accept either two or three positional coordinate arguments, and `Axis3` knows to interpret them in 3-D space.

Column slices of the 3-D matrices:

`cp = bz.points` is the 5×3 control point matrix (stored directly in the struct).

`cp[:,1] = [0, 0, 4, 4, 5]` — x coords

`cp[:,2] = [0, 4, 0, 4, 4]` — y coords

`cp[:,3] = [1, 1, 1, 1, 1]` — z coords (all 1)

These three vectors are passed to `lines!` to draw the control polygon in 3-D space.

5.3. 3-D text annotations

```
# Control point labels
for (i, (name, (dx, dy, dz))) in enumerate(zip(CTRL_NAMES, CTRL_OFFSETS))
    text!(ax,
        cp[i,1]+dx, cp[i,2]+dy, cp[i,3]+dz,
        text     = name,
        color    = COLOR_CTRL,
        fontsize = 9,
        font     = :bold,
    )
end
```

The label loop now destructures a 3-tuple `(dx, dy, dz)` from `CTRL_OFFSETS` instead of a 2-tuple. `text!` is called with three position arguments: `cp[i,1]+dx`, `cp[i,2]+dy`, `cp[i,3]+dz`.

```
# Highlighted point labels
for (i, u) in enumerate(u_highlight)
    pt = hpts[i, :]
    text!(ax,
        pt[1]+0.08, pt[2]+0.08, pt[3]+0.03,
        text     = @sprintf("u=%.1f", u),
        color    = COLOR_LABEL,
        fontsize = 7,
    )
end
```

Rather than calling `bq_point(bz, u)` again, this loop reads directly from the pre-computed `hpts` matrix using row-index access: `hpts[i, :]` gives the i-th row as a length-3 vector `[x, y, z]`. `pt[1]`, `pt[2]`, `pt[3]` are then the individual coordinates.

The format string `"u=%.1f"` shows one decimal place (`0.1` not `0.10`) — appropriate for labels on a dense plot with 11 points.

5.4. `Label` — a figure-level text overlay

```
eq = "x(u) = 13u^4 - 32u^3 + 24u^2\n" *
     "y(u) = -28u^4 + 64u^3 - 48u^2 + 16u\n" *
     "z(u) = 1"
Label(fig[0, 1],
```

```
    eq,
    fontsize  = 9,
    font      = "Courier New",
    halign    = :left,
    padding   = (8, 8, 4, 4),
    tellwidth = false,
)
```

This is the most new concept in the script.

`Label` vs `text!`: `text!(ax, x, y, z, ...)` places text at a *data coordinate* inside the axis — the position is measured in the same units as the plot data (x, y, z).

`Label(fig[...], text, ...)` places text at a *layout position* in the figure grid — it is not inside any axis at all.

`fig[0, 1]`: Grid row 0, column 1. Row 0 is *above* the main axis at `fig[1, 1]`. Makie's grid layout allows row 0 (and negative rows) as positions above the first content row. This places the equation text above the 3-D plot area, not overlapping it.

`font = "Courier New"`: A monospace font name as a string. For equation text, monospace spacing makes the columns align naturally without LaTeX.

`halign = :left`: Horizontal alignment of the text within its grid cell. `:left` means the text starts from the left edge of the cell — matching the top-left position visible in the plot.

`padding = (8, 8, 4, 4)`: Padding in pixels: `(left, right, top, bottom)`. Adds a small margin around the label so it doesn't sit flush against the figure edge.

`tellwidth = false`: When `true` (the default), a `Label` would influence how wide its grid column is — stretching the column to fit the text. Setting `false` lets the column stay its natural width (set by the axis), and the label simply does not affect layout calculations. Without this, the equation text might push the 3-D axis to one side.

> **The equation string:**
>
> `"x(u) = 13u^4 - 32u^3 + 24u^2"`
>
> This is a plain string — no LaTeX, no maths rendering. The `^` is a literal caret character (as typed on the keyboard), not a LaTeX superscript. CairoMakie renders it as-is, which is why the output shows `13u^4` rather than $13u^4$. This is acceptable for a quick annotation; for publication-quality equations you would use a different approach.

5.5. Legend

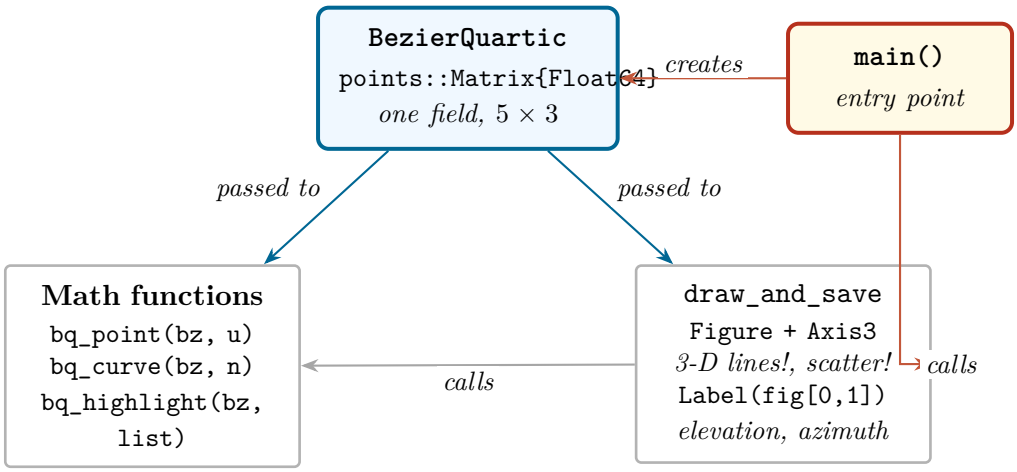
```
axislegend(ax,
    position        = :lt,
    labelsize       = 10,
    framecolor      = (:black, 0.3),
    backgroundcolor = (:white, 0.8),
)
```

`position = :lt` means left-top. In the previous scripts `:rt` (right-top) was used. Here `:lt` is chosen because the 3-D camera angle leaves the upper-left region of the plot relatively empty, while the upper-right has axis labels and the curve.

Makie legend position symbols:

`:lt` = left-top `:rt` = right-top `:lb` = left-bottom `:rb` = right-bottom `:ct` = centre-top `:cb` = centre-bottom

6. Type Map



7. Entry Point

```julia
function main()
    bz = BezierQuartic(CONTROL_POINTS)
    draw_and_save(bz, U_HIGHLIGHT, "21-bezier-quartic-makie.png")
end

main()
```

How to run:

```julia
julia 21-bezier-quartic-makie.jl
```

Saves `21-bezier-quartic-makie.png` and opens a window. Press Enter to close.